

Fetch API & Backend Communication - Notes

1. What is Fetch API?

Fetch API is used to **send requests from frontend to backend** and receive responses.

Frontend → sends request

Backend → processes request → returns response

This is mainly used in **CRUD applications**.

2. CRUD Operations & HTTP Methods

Operation	Method	Purpose
Create	POST	Send data to database
Read	GET	Fetch data from database
Update (full)	PUT	Update all fields
Update (partial)	PATCH	Update specific field
Delete	DELETE	Remove data

PUT vs PATCH

- **PUT** → update all fields (email + password)
 - **PATCH** → update only one field (e.g. password only)
-

3. Postman

Postman is used to **test API endpoints** without frontend.

- Check request
 - Check response
 - Debug backend easily
-

4. Status Codes (Very Important)

Success

- **200 – 299** → Request successful

Client Errors

- **400 – 499** → Client-side problem

Example:

- Missing data
- Validation failed
- Wrong input format

Server Errors

- **500 – 599** → Backend or database issue
-

5. Backend Constraints (Schema Validation)

Backend uses **schema** to validate data. Example:

- Email format

- Password length If data does not match schema → **400 error**

6. Fetch API Requirements

1. URL

Must match backend route exactly.

Example backend:

```
app.use("/api/login", require("./route"));
```

Frontend must use:

```
/api/login
```

2. Method

Defines what action to perform:

```
method: "POST"
```

3. Headers

Tell backend what type of data is being sent.

```
headers: {  
  "Content-Type": "application/json"  
}
```

4. Body (Payload)

Data entered by user.

```
body: JSON.stringify({ email, password })
```

7. Why JSON.stringify()?

- JavaScript objects **cannot travel over network**
- Network only understands **string**
- So we convert object → string

Frontend & Backend both use JSON But communication happens via **string**

8. Example Fetch Code

```
const res = await fetch("https://localhost:5000/api/signup", {  
  method: "POST",  
  headers: {  
    "Content-Type": "application/json",  
  },  
  body: JSON.stringify({ email, password }),  
});
```

9. Handling Response

```
const recData = await res.json();

if (!res.ok) {
  throw new Error(recData.message);
}
```

Why `throw` is used?

- Backend/database errors are complex
- We manually throw error so `catch` can handle it easily

10. Clearing Input Fields

After successful signup:

```
setEmail("");
setPassword("");
```

This clears **fiber nodes (state values)** and updates UI.

11. Loading State

```
setIsLoading(true);
```

Used before request starts

```
finally {
  setIsLoading(false);
}
```

Ensures loading stops in **both success & error**

12. Error Handling with useState

```
const [error, setError] = useState(null);
```

Inside catch:

```
catch (error) {
  setError(error.message);
}
```

13. Display Error (Short Circuiting)

```
<p>{error} && <span>{error}</span></p>
```

Short Circuiting Rules

- `&&` → if first is false, second won't run
- `||` → if first is true, second won't run

Used to **conditionally render UI**

14. Complete Signup Flow Summary

1. User submits form
 2. Fetch sends POST request
 3. Backend validates via schema
 4. Response returned with status code
 5. Success → clear fields
 6. Error → display message
 7. Loading handled using finally
-

15. Port Number (5000)

- Depends on your system
 - One backend can handle:
 - POST
 - GET
 - PUT
 - PATCH
 - DELETE on same port
-

Final Key Points

- Fetch connects frontend to backend
 - Always check `res.ok`
 - Use `JSON.stringify()`
 - Use `try-catch-finally`
 - Display errors using state
 - Match frontend route with backend route exactly
-